



ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Gym Membership Management System**” is submitted by

M PRANEETH	(25B11CS587)
L V NITHEESH PATRUDU	(25B11CS515)
D MAHESH BABU	(25B11CS215)
NAVA JEEVAN	(25B11CS969)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027

Project Guide

Mrs.K.Vydehi ,M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli,Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “Gym Membership Management System” is original and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfillment of the requirements for the award of the B.Tech. Degree in Computer Science and Engineering. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date: 01-10-26

By:

M PRANEETH	(25B11CS587)
L V NITHEESH PATRUDU	(25B11CS515)
D MAHESH BABU	(25B11CS215)
NAVA JEEVAN	(25B11CS969)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of our Gym Membership Management System project.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor**, **Dr. Hari Jyothula, Associate Professor**, and **Dr. Pennada Siva Satya Prasad**, Assistant Professor, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, Dr. Phani Sridhar Addepalli, Assistant Professor, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M. V. Rajesh**, Associate Dean, School of Computing, for his encouragement and support during the course of our project.

We would like to express our sincere gratitude to Dr. G. Suresh, Registrar; **Dr. A. Ramesh**, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); **Dr. M. B. Srinivas**, Vice-Chancellor; **Dr. M. Sreenivasa Reddy**, Deputy Pro Chancellor and Management, Aditya University, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

1. Cover Page.....	
2. Certificate	1
3. Declaration.....	2
4. Acknowledgement	3
5. Table of Contents.....	4
6. Abstract	5
7. List of Figures	6
8. List of Tables	6
9. Acronyms	7
10. Introduction.....	8
11. System Analysis.....	10
12. Database Design.....	12
13. ER Diagram	14
14. Relational Schema.....	15
15. Table Design.....	16
16. Normalization	22
17. SQL Implementation.....	24
18. SQL Queries & Screenshots	27
19. Conclusion	40
20. Future Enhancements	41
21. References	42

ABSTRACT

The Gym Membership Management System is a database application designed to efficiently manage the major operations of a gym or fitness centre through a centralized relational database. The system stores and manages information related to members, trainers, membership plans, memberships, payments, and daily attendance.

The system uses primary keys, foreign keys, unique constraints, and NOT NULL constraints to maintain data integrity and consistency. The database organizes related information into six separate tables, thereby reducing data redundancy and preventing insertion, deletion, and update anomalies.

The database is normalized up to Third Normal Form (3NF). The system supports efficient retrieval of gym information through SQL queries involving selection, filtering, joins, aggregation, subqueries, sorting, views, and reporting.

The proposed Gym Membership Management System provides a structured database foundation for managing members, trainers, membership plans, subscriptions, payments, and attendance. It can be further extended into a complete gym application with online registration, automated renewal reminders, biometric attendance, and reporting modules.

LIST OF FIGURES

No.	Title	Page No.
Figure 1	ER Diagram of Gym Membership Management System	14
Figure 2	Relational Schema Diagram of the Gym Membership Management System	15

LIST OF TABLES

No.	Title	Page No.
Table 1	Major Database Entities	12
Table 2	Members Table Structure	16
Table 3	Trainers Table Structure	16
Table 4	Membership_Plans Table Structure	16
Table 5	Memberships Table Structure	17
Table 6	Payments Table Structure	17
Table 7	Attendance Table Structure	17
Table 8	Keys and Constraints	18
Table 9	Sample Data Records	18

ACRONYMS

DBMS - Database Management System

ER - Entity Relationship

SQL - Structured Query Language

DDL - Data Definition Language

DML - Data Manipulation Language

DQL - Data Query Language

DCL - Data Control Language

TCL - Transaction Control Language

PK - Primary Key

FK - Foreign Key

1NF - First Normal Form

2NF - Second Normal Form

3NF - Third Normal Form

UPI - Unified Payments Interface

1. INTRODUCTION

1.1 Brief Information about the Project:

The Gym Membership Management System is a database application designed to manage and organize the major activities of a gym using a relational database. The system maintains information about members, trainers, membership plans, memberships, payments, and attendance in a structured manner.

The system is implemented using Oracle Database and uses relational database concepts such as primary keys, foreign keys, constraints, joins, aggregate functions, subqueries, views, and normalization.

The database is designed to reduce data redundancy and improve data consistency by dividing information into appropriate relations. Member details are kept in MEMBERS, trainer details in TRAINERS, and plan details in MEMBERSHIP_PLANS, while the MEMBERSHIPS table links a member with a plan and a trainer for a particular period. Payments and attendance are recorded in separate tables so that each fact is stored only once.

1.2 Motivation and Contribution of Project:

A gym handles different types of information every day, including member details, trainer assignments, plan fees, membership validity, fee payments, and daily attendance. Maintaining these details in registers or unstructured spreadsheets can result in duplicate information, missed renewals, inconsistent records, and difficulty in retrieving the required information.

The motivation for developing the Gym Membership Management System is to provide a structured database that can store gym information systematically. The use of a relational database allows related information to be connected through keys and retrieved quickly using SQL queries.

1.3 Objectives of the Project:

The main objectives are:

- To design a relational database for gym membership management.
- To store member details and trainer details.
- To maintain different membership plans with their duration and fee.
- To record the membership taken by each member, together with the assigned trainer and validity period.
- To record payment information for each membership.
- To maintain daily attendance of members.
- To reduce data redundancy through normalization.
- To maintain data integrity using primary keys, foreign keys, and constraints.

- To generate useful reports such as revenue, active memberships, and attendance summaries using SQL.

1.4 Scope of the Project:

The scope of the Gym Membership Management System includes:

- Member management
- Trainer management
- Membership plan management
- Membership (subscription) management
- Payment management
- Attendance management
- Revenue analysis
- Plan and payment-method analysis
- Member attendance analysis
- Trainer assignment analysis

2. SYSTEM ANALYSIS

2.1 Existing System:

In a basic/manual gym system, information may be maintained using registers, notebooks, spreadsheets, or disconnected records.

Common problems include:

- Manual record maintenance.
- Duplicate member information.
- Difficulty tracking membership expiry and renewals.
- Difficulty tracking fee payments.
- Difficulty recording and analyzing attendance.
- Difficulty tracking which trainer is assigned to which member.
- Higher possibility of data-entry errors.
- Difficulty generating consolidated reports.
- Lack of proper relationships between records.

2.2 Proposed System:

The proposed Gym Membership Management System uses an Oracle relational database to organize gym information into related tables.

The system consists of six tables:

1. Members
2. Trainers
3. Membership_Plans
4. Memberships
5. Payments
6. Attendance

Primary keys uniquely identify records, while foreign keys establish relationships between tables.

The proposed system supports:

- Member management
- Trainer management
- Membership plan management
- Membership and payment management
- Attendance management
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements:

Module	Functional Requirement
Member Management	Add, update, search and display member details
Trainer Management	Maintain trainer details and specialization
Plan Management	Maintain membership plans, durations and fees
Membership Management	Record the plan and trainer chosen by a member, with start date, end date and status
Payment Management	Record payments made for memberships
Attendance Management	Record daily Present/Absent attendance of members
Reporting	Generate gym reports using SQL queries

2.4 Non-Functional Requirements:

Performance:

The database should provide efficient query execution for normal gym workloads.

Integrity:

Primary keys and foreign keys should prevent invalid references and duplicate primary-key values.

Security:

Database privileges should be controlled so that only authorized users can modify important gym records.

Reliability:

Transactions should maintain a consistent state of the database.

Scalability:

The database should be capable of handling an increasing number of members, memberships, payments and attendance records.

Usability:

The database structure and reports should be understandable to gym administrators.

2.5 Requirements Specification

Category	Requirement
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	Adequate storage for Oracle Database
Operating System	Windows 10/11 or equivalent
Database	Oracle Database
Query Language	SQL
Development Environment	Oracle SQL Developer / SQL*Plus

3. DATABASE DESIGN

3.1 Major Entities:

Table 1: Major Database Entities

Entity	Description
Members	Stores member information such as member ID, name, age, gender, phone number, and joining date.
Trainers	Stores trainer information such as trainer ID, name, specialization, and phone number.
Membership_Plans	Stores plan information such as plan name, duration in months, and fee.
Memberships	Stores the plan and trainer chosen by a member, along with the start date, end date, and status of the membership.
Payments	Stores payment information such as date, amount, and payment method associated with a membership.
Attendance	Stores the daily attendance status (Present/Absent) of members.

3.2 ER Model:

The ER model of the Gym Membership Management System contains six major entities. MEMBERSHIPS acts as the central (associative) entity that connects members, plans and trainers.

The major relationships are:

- Member subscribes to Membership.
- Membership is based on a Membership Plan.
- Trainer is assigned to Membership.
- Membership has Payments.
- Member marks Attendance.

3.3 Relationships:

Relationship	Description
Members → Memberships	One member can hold one or more memberships (1:N)
Membership_Plans → Memberships	One plan can be chosen in many memberships (1:N)
Trainers → Memberships	One trainer can be assigned to many memberships (1:N)
Memberships → Payments	One membership can have one or more payments (1:N)
Members → Attendance	One member can have many attendance records (1:N)
Members ↔ Membership_Plans	Many-to-many relationship resolved through Memberships
Members ↔ Trainers	Many-to-many relationship resolved through Memberships

3.4 Business Rules and Assumptions:

- Each member, trainer, plan, membership, payment and attendance record is identified by a unique numeric ID.
- Phone numbers of members and of trainers must be unique.
- A membership belongs to exactly one member, one plan and one trainer.
- The status of a membership is stored as Active or Expired.
- The attendance status of a member on a date is stored as Present or Absent.
- Each payment is linked to the membership for which it was made.

4. ER DIAGRAM

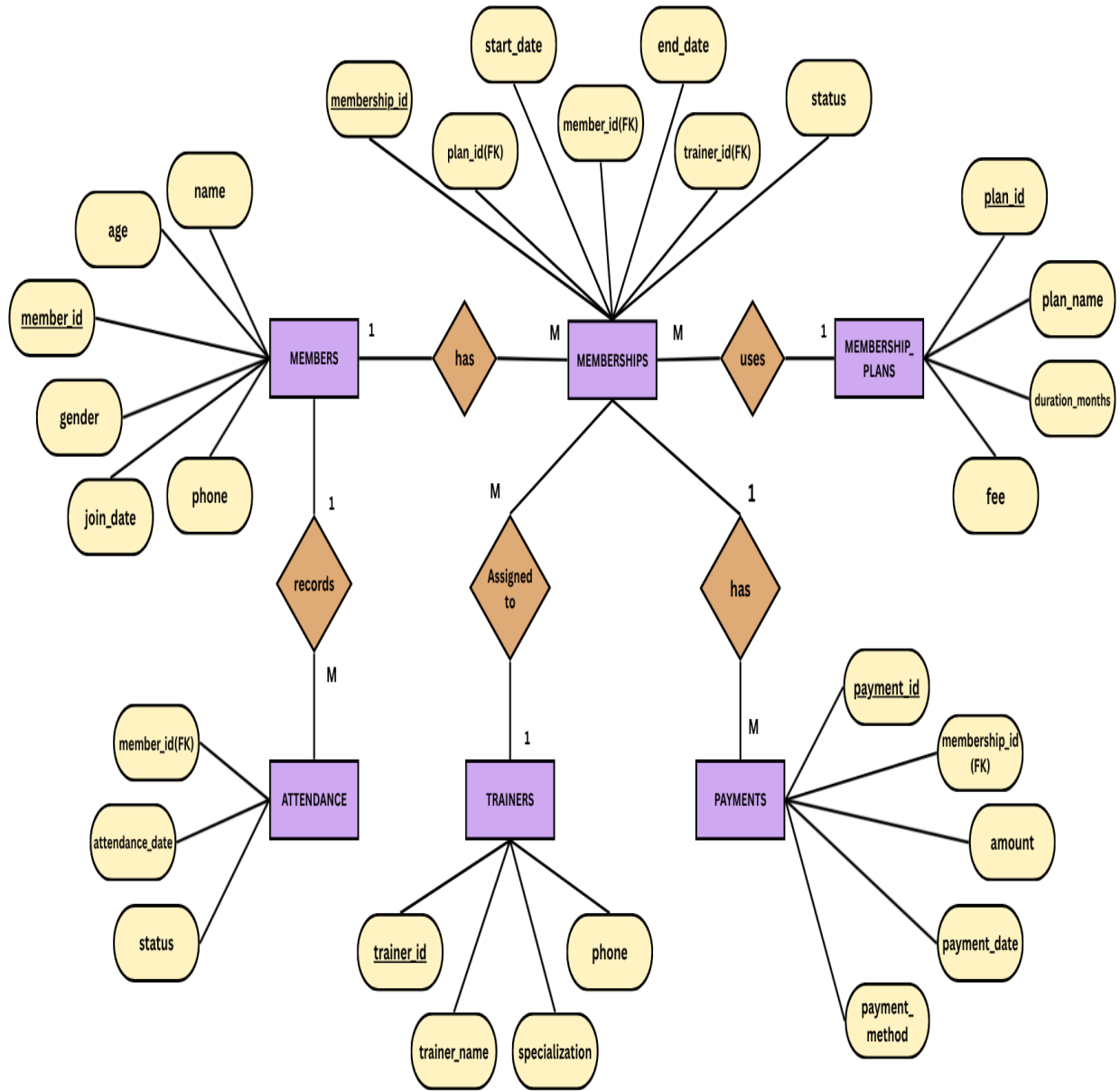


Figure 1: ER Diagram of Gym Membership Management System

Notation: rectangle = entity, diamond = relationship, ellipse = attribute, underlined attribute = primary key, 1 / N = cardinality.

5. RELATIONAL SCHEMA

RELATION	RELATIONAL SCHEMA
1. MEMBERS	member_id (PK), name, age, gender, phone UNIQUE, join_date
2. TRAINERS	trainer_id (PK), trainer_name, specialization, phone UNIQUE
3. MEMBERSHIP_PLANS	plan_id (PK), plan_name, duration_months, fee
4. MEMBERSHIPS	membership_id (PK), member_id (FK), plan_id (FK), trainer_id (FK), start_date, end_date, status
5. PAYMENTS	payment_id (PK), membership_id (FK), payment_date, amount, payment_method
6. ATTENDANCE	attendance_id (PK), member_id (FK), attendance_date, status

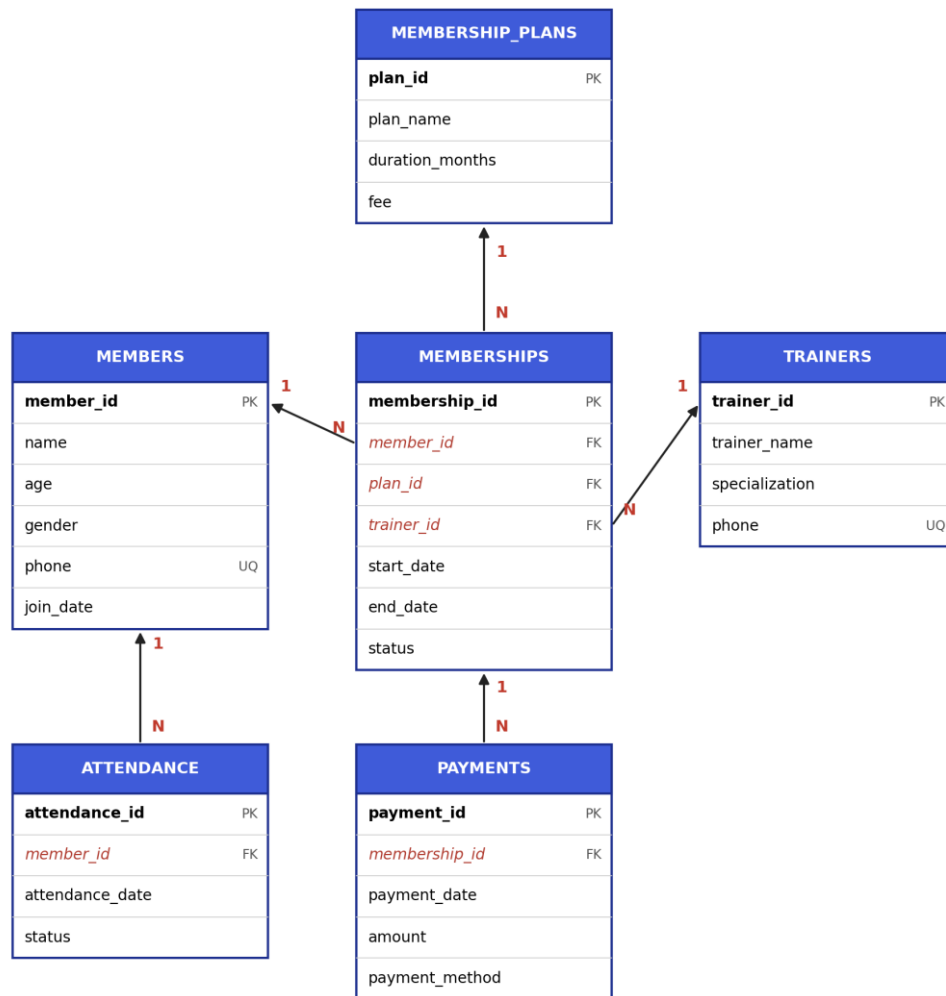


Figure 2: Relational Schema Diagram of the Gym Membership Management System

6. TABLE DESIGN

6.1 MEMBERS Table

Table 2: Members Table Structure

Attribute	Data Type	Constraint	Description
member_id	NUMBER	PK	Unique member identifier
name	VARCHAR2(50)	NOT NULL	Member name
age	NUMBER	-	Member age
gender	VARCHAR2(10)	-	Member gender
phone	VARCHAR2(15)	UNIQUE	Member phone number
join_date	DATE	NOT NULL	Date the member joined the gym

6.2 TRAINERS Table

Table 3: Trainers Table Structure

Attribute	Data Type	Constraint	Description
trainer_id	NUMBER	PK	Unique trainer identifier
trainer_name	VARCHAR2(50)	NOT NULL	Trainer name
specialization	VARCHAR2(50)	-	Area of specialization
phone	VARCHAR2(15)	UNIQUE	Trainer phone number

6.3 MEMBERSHIP_PLANS Table

Table 4: Membership_Plans Table Structure

Attribute	Data Type	Constraint	Description
plan_id	NUMBER	PK	Unique plan identifier
plan_name	VARCHAR2(30)	NOT NULL	Name of the plan
duration_months	NUMBER	NOT NULL	Plan duration in months
fee	NUMBER(10,2)	NOT NULL	Plan fee

6.4 MEMBERSHIPS Table

Table 5: Memberships Table Structure

Attribute	Data Type	Constraint	Description
membership_id	NUMBER	PK	Unique membership identifier
member_id	NUMBER	FK	Member who took the membership
plan_id	NUMBER	FK	Plan chosen
trainer_id	NUMBER	FK	Trainer assigned
start_date	DATE	NOT NULL	Membership start date
end_date	DATE	NOT NULL	Membership end date
status	VARCHAR2(20)	NOT NULL	Active / Expired

6.5 PAYMENTS Table

Table 6: Payments Table Structure

Attribute	Data Type	Constraint	Description
payment_id	NUMBER	PK	Unique payment identifier
membership_id	NUMBER	FK	Membership for which payment is made
payment_date	DATE	NOT NULL	Date of payment
amount	NUMBER(10,2)	NOT NULL	Amount paid
payment_method	VARCHAR2(20)	-	Cash, UPI, Card etc.

6.6 ATTENDANCE Table

Table 7: Attendance Table Structure

Attribute	Data Type	Constraint	Description
attendance_id	NUMBER	PK	Unique attendance identifier
member_id	NUMBER	FK	Member whose attendance is recorded
attendance_date	DATE	NOT NULL	Date of attendance
status	VARCHAR2(15)	NOT NULL	Present / Absent

6.7 KEYS AND CONSTRAINTS

Table 8: Keys and Constraints

Constraint	Applied To	Purpose
Primary Key	All six tables	Uniquely identifies each record
Foreign Key	Memberships.member_id	References Members
Foreign Key	Memberships.plan_id	References Membership_Plans
Foreign Key	Memberships.trainer_id	References Trainers
Foreign Key	Payments.membership_id	References Memberships
Foreign Key	Attendance.member_id	References Members
Unique	Members.phone, Trainers.phone	Prevents duplicate phone numbers
NOT NULL	Required attributes	Prevents missing required values

6.8 SAMPLE DATA RECORDS

Each table is populated with 15 records. The records are interconnected so that join, group-by, having and subquery queries return meaningful results.

Table 9: Sample Data Records

MEMBERS (15 records)

member_id	name	age	gender	phone	join_date
1	Rahul	21	Male	9000000001	05-JAN-26
2	Priya	22	Female	9000000002	10-JAN-26
3	Arjun	20	Male	9000000003	15-JAN-26
4	Sneha	23	Female	9000000004	20-JAN-26
5	Kiran	24	Male	9000000005	01-FEB-26
6	Ananya	21	Female	9000000006	05-FEB-26
7	Vikram	25	Male	9000000007	10-FEB-26
8	Neha	22	Female	9000000008	15-FEB-26
9	Rohit	26	Male	9000000009	20-FEB-26
10	Pooja	23	Female	9000000010	01-MAR-26
11	Aditya	21	Male	9000000011	05-MAR-26
12	Divya	24	Female	9000000012	10-MAR-26
13	Suresh	27	Male	9000000013	15-MAR-26
14	Meena	22	Female	9000000014	20-MAR-26
15	Karthik	25	Male	9000000015	25-MAR-26

TRAINERS (15 records)

trainer_id	trainer_name	specialization	phone
1	Amit	Strength Training	9100000001
2	Neeraj	Cardio	9100000002
3	Ravi	Weight Loss	9100000003
4	Kavya	Yoga	9100000004
5	Manoj	Bodybuilding	9100000005
6	Shreya	Fitness Training	9100000006
7	Arvind	CrossFit	9100000007
8	Nisha	Yoga	9100000008
9	Sanjay	Strength Training	9100000009
10	Deepa	Cardio	9100000010
11	Vijay	Weight Loss	9100000011
12	Pallavi	Fitness Training	9100000012
13	Ramesh	Bodybuilding	9100000013
14	Swathi	CrossFit	9100000014
15	Tarun	Strength Training	9100000015

MEMBERSHIP_PLANS (15 records)

plan_id	plan_name	duration_months	fee
1	Monthly Basic	1	1000
2	Monthly Premium	1	1500
3	Quarterly Basic	3	2700
4	Quarterly Premium	3	4000
5	Half Year Basic	6	5000
6	Half Year Premium	6	7000
7	Annual Basic	12	9000
8	Annual Premium	12	12000
9	Student Monthly	1	800
10	Student Quarterly	3	2200
11	Couple Monthly	1	2500
12	Couple Quarterly	3	6500
13	Personal Training	3	9000
14	Elite Annual	12	15000
15	Corporate Annual	12	13000

MEMBERSHIPS (15 records)

membership_id	member_id	plan_id	trainer_id	start_date	end_date	status
1	1	7	1	05-JAN-26	04-JAN-27	Active
2	2	8	2	10-JAN-26	09-JAN-27	Active
3	3	3	3	15-JAN-26	14-APR-26	Expired
4	4	4	4	20-JAN-26	19-APR-26	Expired
5	5	5	5	01-FEB-26	31-JUL-26	Active
6	6	6	6	05-FEB-26	04-AUG-26	Active
7	7	9	7	10-FEB-26	09-MAR-26	Expired
8	8	10	8	15-FEB-26	14-MAY-26	Active
9	9	14	9	20-FEB-26	19-FEB-27	Active
10	10	1	10	01-MAR-26	31-MAR-26	Expired
11	11	2	11	05-MAR-26	04-APR-26	Expired
12	12	12	12	10-MAR-26	09-JUN-26	Active
13	13	13	13	15-MAR-26	14-JUN-26	Active
14	14	11	14	20-MAR-26	19-APR-26	Expired
15	15	15	15	25-MAR-26	24-MAR-27	Active

PAYMENTS (15 records)

payment_id	membership_id	payment_date	amount	payment_method
1	1	05-JAN-26	9000	UPI
2	2	10-JAN-26	12000	Card
3	3	15-JAN-26	2700	Cash
4	4	20-JAN-26	4000	UPI
5	5	01-FEB-26	5000	Card
6	6	05-FEB-26	7000	UPI
7	7	10-FEB-26	800	Cash
8	8	15-FEB-26	2200	UPI
9	9	20-FEB-26	15000	Card
10	10	01-MAR-26	1000	Cash
11	11	05-MAR-26	1500	UPI
12	12	10-MAR-26	6500	Card
13	13	15-MAR-26	9000	UPI
14	14	20-MAR-26	2500	Cash
15	15	25-MAR-26	13000	Card

ATTENDANCE (15 records)

attendance_id	member_id	attendance_date	status
1	1	01-MAR-26	Present
2	2	01-MAR-26	Present
3	3	01-MAR-26	Absent
4	4	01-MAR-26	Present
5	5	02-MAR-26	Present
6	6	02-MAR-26	Absent
7	7	02-MAR-26	Present
8	8	03-MAR-26	Present
9	9	03-MAR-26	Present
10	10	03-MAR-26	Absent
11	11	04-MAR-26	Present
12	12	04-MAR-26	Present
13	13	04-MAR-26	Absent
14	14	05-MAR-26	Present
15	15	05-MAR-26	Present

7. NORMALIZATION

Normalization organizes data into related tables to reduce redundancy and avoid insertion, deletion, and update anomalies. The design of the Gym Membership Management System is presented through the stages UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF.

7.1 Unnormalized Form (UNF):

A single gym record might contain Member_ID, Member_Name, Age, Gender, Phone, Join_Date, Plan_Name, Duration_Months, Fee, Trainer_Name, Specialization, Start_Date, End_Date, Status, multiple Payment_Date / Amount / Payment_Method values, and multiple Attendance_Date / Attendance_Status values in one record. The repeating payment and attendance groups, along with repeated member, plan and trainer information, create data redundancy and update anomalies. This unstructured form represents the Unnormalized Form (UNF) of the Gym Membership Management System.

7.2 First Normal Form (1NF):

In 1NF, repeating groups are removed and each attribute contains atomic values. Each payment and each attendance entry is represented as a separate row with its corresponding Member_ID or Membership_ID. Member, plan and trainer details may still be repeated across many rows. This removes repeating groups, but data redundancy still exists.

7.3 Second Normal Form (2NF):

In 2NF, partial dependencies on a composite key are removed. In the Gym Membership Management System, Member_ID determines member-specific attributes such as Name, Age, Gender, Phone and Join_Date; Plan_ID determines Plan_Name, Duration_Months and Fee; Trainer_ID determines Trainer_Name, Specialization and Phone; Membership_ID determines the member, plan, trainer, Start_Date, End_Date and Status; Payment_ID determines Payment_Date, Amount and Payment_Method; and Attendance_ID determines Attendance_Date and Status. These dependencies motivate separating the MEMBERS, TRAINERS, MEMBERSHIP_PLANS, MEMBERSHIPS, PAYMENTS and ATTENDANCE relations.

7.4 Third Normal Form (3NF):

In 3NF, transitive dependencies are removed by separating attributes that depend on other non-key attributes. For example, in a combined membership record, Membership_ID $\rightarrow$ Plan_ID $\rightarrow$ (Plan_Name, Duration_Months, Fee) and Membership_ID $\rightarrow$ Trainer_ID $\rightarrow$ (Trainer_Name, Specialization) are transitive dependencies. They are removed by storing plan details only in MEMBERSHIP_PLANS and trainer details only in TRAINERS; MEMBERSHIPS keeps only the keys that refer to them.

As a result, in every relation each non-key attribute depends on the key, the whole key, and nothing but the key. The end_date of a membership is stored explicitly so that the

exact validity period of a membership is preserved even if the plan definition changes later.

Relation	Reason for Separation
MEMBERS	Stores member-specific details such as name, age, gender, phone and join date.
TRAINERS	Stores trainer-specific details such as name, specialization and phone.
MEMBERSHIP_PLANS	Stores plan details such as plan name, duration and fee.
MEMBERSHIPS	Connects a member with a plan and a trainer and stores the start date, end date and status.
PAYMENTS	Stores payment date, amount and method and references the membership.
ATTENDANCE	Stores attendance date and status and references the member.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

Because of the foreign-key relationships, the tables are created in the following order:
Members → Trainers → Membership_Plans → Memberships → Payments → Attendance.

Members Table

```
CREATE TABLE Members (  
    member_id NUMBER PRIMARY KEY,  
    name VARCHAR2(50) NOT NULL,  
    age NUMBER,  
    gender VARCHAR2(10),  
    phone VARCHAR2(15) UNIQUE,  
    join_date DATE NOT NULL  
);
```

Trainers Table

```
CREATE TABLE Trainers (  
    trainer_id NUMBER PRIMARY KEY,  
    trainer_name VARCHAR2(50) NOT NULL,  
    specialization VARCHAR2(50),  
    phone VARCHAR2(15) UNIQUE  
);
```

Membership_Plans Table

```
CREATE TABLE Membership_Plans (  
    plan_id NUMBER PRIMARY KEY,  
    plan_name VARCHAR2(30) NOT NULL,  
    duration_months NUMBER NOT NULL,  
    fee NUMBER(10,2) NOT NULL  
);
```

Memberships Table

```
CREATE TABLE Memberships (  
    membership_id NUMBER PRIMARY KEY,  
    member_id NUMBER,  
    plan_id NUMBER,  
    trainer_id NUMBER,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    status VARCHAR2(20) NOT NULL,  
    FOREIGN KEY (member_id) REFERENCES Members(member_id),  
    FOREIGN KEY (plan_id) REFERENCES Membership_Plans(plan_id),  
    FOREIGN KEY (trainer_id) REFERENCES Trainers(trainer_id)  
);
```

Payments Table

```
CREATE TABLE Payments (  
    payment_id NUMBER PRIMARY KEY,  
    membership_id NUMBER,  
    payment_date DATE NOT NULL,  
    amount NUMBER(10,2) NOT NULL,  
    payment_method VARCHAR2(20),  
    FOREIGN KEY (membership_id) REFERENCES Memberships(membership_id)  
);
```

Attendance Table

```
CREATE TABLE Attendance (  
    attendance_id NUMBER PRIMARY KEY,  
    member_id NUMBER,  
    attendance_date DATE NOT NULL,  
    status VARCHAR2(15) NOT NULL,  
    FOREIGN KEY (member_id) REFERENCES Members(member_id)  
);
```

8.2 DML – Data Manipulation Language

The database is populated using INSERT statements, 15 records in each table (90 records in total). For example:

```
INSERT INTO Members VALUES (1, 'Rahul', 21, 'Male', '90000000001',  
DATE '2026-01-05');  
INSERT INTO Trainers VALUES (1, 'Amit', 'Strength Training',  
'91000000001');  
INSERT INTO Membership_Plans VALUES (1, 'Monthly Basic', 1, 1000);  
INSERT INTO Memberships VALUES (1, 1, 7, 1, DATE '2026-01-05', DATE  
'2027-01-04', 'Active');  
INSERT INTO Payments VALUES (1, 1, DATE '2026-01-05', 9000, 'UPI');  
INSERT INTO Attendance VALUES (1, 1, DATE '2026-03-01', 'Present');
```

After all records are inserted, the transaction is saved and the row count of every table is verified:

```
COMMIT;  
  
SELECT COUNT(*) FROM Members;  
SELECT COUNT(*) FROM Trainers;  
SELECT COUNT(*) FROM Membership_Plans;  
SELECT COUNT(*) FROM Memberships;  
SELECT COUNT(*) FROM Payments;  
SELECT COUNT(*) FROM Attendance;
```

Each of the six queries returns 15.

8.3 DQL – Data Query Language

1.

```
SELECT MIN(fee) AS minimum_fee,  
       MAX(fee) AS maximum_fee  
FROM Membership_Plans;
```

2.

```
SELECT duration_months,  
       COUNT(*) AS number_of_plans  
FROM Membership_Plans  
GROUP BY duration_months  
ORDER BY duration_months;
```

3.

```
SELECT member_id, name, age, gender  
FROM Members  
WHERE name LIKE 'A%';
```

4.

```
SELECT plan_id, plan_name, fee  
FROM Membership_Plans  
WHERE fee > 5000  
ORDER BY plan_id;
```

5.

```
SELECT m.member_id, m.name, ms.start_date,  
       ms.end_date, ms.status  
FROM Members m  
JOIN Memberships ms  
ON m.member_id = ms.member_id  
ORDER BY m.member_id;
```

8.4 TCL – Transaction Control Language

The COMMIT statement makes the inserted data permanent in the database. ROLLBACK can be used to undo uncommitted changes if an error is made while entering data.

9. SQL QUERIES & SCREENSHOTS

The following query set covers selection, filtering, joins, aggregation, grouping with HAVING, subqueries, sorting, and views on the final 90-record project database (15 records in each of the six tables). The output of each query is shown below it.

1. Minimum and Maximum Plan Fee

```
SELECT MIN(fee) AS minimum_fee,  
       MAX(fee) AS maximum_fee  
FROM Membership_Plans;
```

Output:

MINIMUM_FEE	MAXIMUM_FEE
800	15000

2. Number of Plans by Duration

```
SELECT duration_months,  
       COUNT(*) AS number_of_plans  
FROM Membership_Plans  
GROUP BY duration_months  
ORDER BY duration_months;
```

Output:

DURATION_MONTHS	NUMBER_OF_PLANS
1	4
3	5
6	2
12	4

3. Members Whose Name Starts With A

```
SELECT member_id, name, age, gender  
FROM Members  
WHERE name LIKE 'A%';
```

Output:

MEMBER_ID	NAME	AGE	GENDER
3	Arjun	20	Male
6	Ananya	21	Female
11	Aditya	21	Male

4. Plans With Fee Greater Than ₹5,000

```
SELECT plan_id, plan_name, fee
FROM Membership_Plans
WHERE fee > 5000
ORDER BY plan_id;
```

Output:

PLAN_ID	PLAN_NAME	FEE
6	Half Year Premium	7000
7	Annual Basic	9000
8	Annual Premium	12000
12	Couple Quarterly	6500
13	Personal Training	9000
14	Elite Annual	15000
15	Corporate Annual	13000

5. Members With Membership Dates and Status

```
SELECT m.member_id, m.name, ms.start_date,
       ms.end_date, ms.status
FROM Members m
JOIN Memberships ms
ON m.member_id = ms.member_id
ORDER BY m.member_id;
```

Output:

MEMBER_ID	NAME	START_DATE	END_DATE	STATUS
1	Rahul	05-JAN-26	04-JAN-27	Active
2	Priya	10-JAN-26	09-JAN-27	Active
3	Arjun	15-JAN-26	14-APR-26	Expired
4	Sneha	20-JAN-26	19-APR-26	Expired
5	Kiran	01-FEB-26	31-JUL-26	Active
6	Ananya	05-FEB-26	04-AUG-26	Active
7	Vikram	10-FEB-26	09-MAR-26	Expired
8	Neha	15-FEB-26	14-MAY-26	Active
9	Rohit	20-FEB-26	19-FEB-27	Active
10	Pooja	01-MAR-26	31-MAR-26	Expired
11	Aditya	05-MAR-26	04-APR-26	Expired
12	Divya	10-MAR-26	09-JUN-26	Active
13	Suresh	15-MAR-26	14-JUN-26	Active
14	Meena	20-MAR-26	19-APR-26	Expired
15	Karthik	25-MAR-26	24-MAR-27	Active

6. Member, Plan and Trainer Details

```
SELECT m.name AS member_name,  
       p.plan_name,  
       t.trainer_name,  
       ms.status  
FROM Memberships ms  
JOIN Members m ON ms.member_id = m.member_id  
JOIN Membership_Plans p ON ms.plan_id = p.plan_id  
JOIN Trainers t ON ms.trainer_id = t.trainer_id  
ORDER BY ms.membership_id;
```

Output:

MEMBER_NAME	PLAN_NAME	TRAINER_NAME	STATUS
Rahul	Annual Basic	Amit	Active
Priya	Annual Premium	Neeraj	Active
Arjun	Quarterly Basic	Ravi	Expired
Sneha	Quarterly Premium	Kavya	Expired
Kiran	Half Year Basic	Manoj	Active
Ananya	Half Year Premium	Shreya	Active
Vikram	Student Monthly	Arvind	Expired
Neha	Student Quarterly	Nisha	Active
Rohit	Elite Annual	Sanjay	Active
Pooja	Monthly Basic	Deepa	Expired
Aditya	Monthly Premium	Vijay	Expired
Divya	Couple Quarterly	Pallavi	Active
Suresh	Personal Training	Ramesh	Active
Meena	Couple Monthly	Swathi	Expired
Karthik	Corporate Annual	Tarun	Active

7. Number of Memberships by Status

```
SELECT status,  
       COUNT(*) AS total_memberships  
FROM Memberships  
GROUP BY status  
ORDER BY status;
```

Output:

STATUS	TOTAL_MEMBERSHIPS
Active	9
Expired	6

8. Total Gym Revenue

```
SELECT SUM(amount) AS total_revenue  
FROM Payments;
```

Output:

TOTAL_REVENUE
91200

9. Revenue by Payment Method

```
SELECT payment_method,  
       COUNT(*) AS no_of_payments,  
       SUM(amount) AS total_revenue  
FROM Payments  
GROUP BY payment_method  
ORDER BY payment_method;
```

Output:

PAYMENT_METHOD	NO_OF_PAYMENTS	TOTAL_REVENUE
Card	5	51500
Cash	4	7000
UPI	6	32700

10. Payment Details With Member Name

```
SELECT p.payment_id, m.name, p.amount,  
       p.payment_method, p.payment_date  
FROM Payments p  
JOIN Memberships ms  
ON p.membership_id = ms.membership_id  
JOIN Members m  
ON ms.member_id = m.member_id  
ORDER BY p.payment_id;
```

Output:

PAYMENT_ID	NAME	AMOUNT	PAYMENT_METHOD	PAYMENT_DATE
1	Rahul	9000	UPI	05-JAN-26
2	Priya	12000	Card	10-JAN-26
3	Arjun	2700	Cash	15-JAN-26
4	Sneha	4000	UPI	20-JAN-26
5	Kiran	5000	Card	01-FEB-26
6	Ananya	7000	UPI	05-FEB-26
7	Vikram	800	Cash	10-FEB-26
8	Neha	2200	UPI	15-FEB-26
9	Rohit	15000	Card	20-FEB-26
10	Pooja	1000	Cash	01-MAR-26
11	Aditya	1500	UPI	05-MAR-26
12	Divya	6500	Card	10-MAR-26
13	Suresh	9000	UPI	15-MAR-26
14	Meena	2500	Cash	20-MAR-26
15	Karthik	13000	Card	25-MAR-26

11. Members Who Paid Using UPI

```
SELECT DISTINCT m.member_id, m.name
FROM Members m
JOIN Memberships ms ON m.member_id = ms.member_id
JOIN Payments p ON ms.membership_id = p.membership_id
WHERE p.payment_method = 'UPI'
ORDER BY m.member_id;
```

Output:

MEMBER_ID	NAME
1	Rahul
4	Sneha
6	Ananya
8	Neha
11	Aditya
13	Suresh

12. Most Expensive Membership Plan

```
SELECT *
FROM Membership_Plans
WHERE fee = (
    SELECT MAX(fee)
    FROM Membership_Plans
);
```

Output:

PLAN_ID	PLAN_NAME	DURATION_MONTHS	FEE
14	Elite Annual	12	15000

13. Number of Trainers in Each Specialization

```
SELECT specialization,
    COUNT(*) AS number_of_trainers
FROM Trainers
GROUP BY specialization
ORDER BY number_of_trainers DESC, specialization;
```

Output:

SPECIALIZATION	NUMBER_OF_TRAINERS
Strength Training	3
Bodybuilding	2
Cardio	2
CrossFit	2
Fitness Training	2
Weight Loss	2
Yoga	2

14. Specializations With More Than Two Trainers

```
SELECT specialization,  
       COUNT(*) AS number_of_trainers  
FROM Trainers  
GROUP BY specialization  
HAVING COUNT(*) > 2;
```

Output:

SPECIALIZATION	NUMBER_OF_TRAINERS
Strength Training	3

15. Members Who Have No Active Membership

```
SELECT member_id, name  
FROM Members  
WHERE member_id NOT IN (  
    SELECT member_id  
    FROM Memberships  
    WHERE status = 'Active'  
)  
ORDER BY member_id;
```

Output:

MEMBER_ID	NAME
3	Arjun
4	Sneha
7	Vikram
10	Pooja
11	Aditya
14	Meena

16. Attendance Count by Status

```
SELECT status,  
       COUNT(*) AS total  
FROM Attendance  
GROUP BY status  
ORDER BY status;
```

Output:

STATUS	TOTAL
Absent	4
Present	11

17. Daily Attendance Summary

```
SELECT attendance_date,  
       COUNT(*) AS total_records,  
       SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END)  
         AS present_count  
FROM Attendance  
GROUP BY attendance_date  
ORDER BY attendance_date;
```

Output:

ATTENDANCE_DATE	TOTAL_RECORDS	PRESENT_COUNT
01-MAR-26	4	3
02-MAR-26	3	2
03-MAR-26	3	2
04-MAR-26	3	2
05-MAR-26	2	2

18. Members Who Were Absent

```
SELECT m.member_id, m.name, a.attendance_date  
FROM Members m  
JOIN Attendance a  
ON m.member_id = a.member_id  
WHERE a.status = 'Absent'  
ORDER BY m.member_id;
```

Output:

MEMBER_ID	NAME	ATTENDANCE_DATE
3	Arjun	01-MAR-26
6	Ananya	02-MAR-26
10	Pooja	03-MAR-26
13	Suresh	04-MAR-26

19. Payments Above the Average Payment

```
SELECT payment_id, membership_id, amount, payment_method
FROM Payments
WHERE amount > (
    SELECT AVG(amount)
    FROM Payments
)sss
ORDER BY payment_id;
```

Output:

PAYMENT_ID	MEMBERSHIP_ID	AMOUNT	PAYMENT_METHOD
1	1	9000	UPI
2	2	12000	Card
6	6	7000	UPI
9	9	15000	Card
12	12	6500	Card
13	13	9000	UPI
15	15	13000	Card

20. Average Payment Amount

```
SELECT AVG(amount) AS average_payment
FROM Payments;
```

Output:

AVERAGE_PAYMENT
6080

21. Members Who Joined in February 2026

```
SELECT member_id, name, join_date
FROM Members
WHERE join_date BETWEEN DATE '2026-02-01'
    AND DATE '2026-02-28'
ORDER BY join_date;
```

Output:

MEMBER_ID	NAME	JOIN_DATE
5	Kiran	01-FEB-26
6	Ananya	05-FEB-26
7	Vikram	10-FEB-26
8	Neha	15-FEB-26
9	Rohit	20-FEB-26

22. Number of Members by Gender

```
SELECT gender,  
       COUNT(*) AS total_members  
FROM Members  
GROUP BY gender  
ORDER BY gender;
```

Output:

GENDER	TOTAL_MEMBERS
Female	7
Male	8

23. Average Age of Members by Gender

```
SELECT gender,  
       ROUND(AVG(age), 2) AS average_age  
FROM Members  
GROUP BY gender  
ORDER BY gender;
```

Output:

GENDER	AVERAGE_AGE
Female	22.43
Male	23.63

24. Top 3 Highest Payments

```
SELECT m.name, p.amount  
FROM Payments p  
JOIN Memberships ms ON p.membership_id = ms.membership_id  
JOIN Members m ON ms.member_id = m.member_id  
ORDER BY p.amount DESC  
FETCH FIRST 3 ROWS ONLY;
```

Output:

NAME	AMOUNT
Rohit	15000
Karthik	13000
Priya	12000

25. Revenue by Plan Duration

```
SELECT pl.duration_months,  
       COUNT(p.payment_id) AS no_of_payments,  
       SUM(p.amount) AS total_revenue  
FROM Payments p  
JOIN Memberships ms ON p.membership_id = ms.membership_id  
JOIN Membership_Plans pl ON ms.plan_id = pl.plan_id  
GROUP BY pl.duration_months  
ORDER BY pl.duration_months;
```

Output:

DURATION_MONTHS	NO_OF_PAYMENTS	TOTAL_REVENUE
1	4	5800
3	5	24400
6	2	12000
12	4	49000

26. Members on Premium Plans

```
SELECT m.member_id, m.name, pl.plan_name, pl.fee  
FROM Members m  
JOIN Memberships ms ON m.member_id = ms.member_id  
JOIN Membership_Plans pl ON ms.plan_id = pl.plan_id  
WHERE pl.plan_name LIKE '%Premium%'  
ORDER BY m.member_id;
```

Output:

MEMBER_ID	NAME	PLAN_NAME	FEE
2	Priya	Annual Premium	12000
4	Sneha	Quarterly Premium	4000
6	Ananya	Half Year Premium	7000
11	Aditya	Monthly Premium	1500

27. Trainers Assigned to Active Memberships

```
SELECT t.trainer_name, t.specialization,  
       m.name AS member_name  
FROM Trainers t  
JOIN Memberships ms ON t.trainer_id = ms.trainer_id  
JOIN Members m ON ms.member_id = m.member_id  
WHERE ms.status = 'Active'  
ORDER BY t.trainer_id;
```

Output:

TRAINER_NAME	SPECIALIZATION	MEMBER_NAME
Amit	Strength Training	Rahul
Neeraj	Cardio	Priya
Manoj	Bodybuilding	Kiran
Shreya	Fitness Training	Ananya
Nisha	Yoga	Neha
Sanjay	Strength Training	Rohit
Pallavi	Fitness Training	Divya
Ramesh	Bodybuilding	Suresh
Tarun	Strength Training	Karthik

28. Members on Plans Longer Than the Average Duration

```

SELECT m.member_id, m.name, pl.plan_name, pl.duration_months
FROM Members m
JOIN Memberships ms ON m.member_id = ms.member_id
JOIN Membership_Plans pl ON ms.plan_id = pl.plan_id
WHERE pl.duration_months > (
    SELECT AVG(duration_months)
    FROM Membership_Plans
)
ORDER BY m.member_id;

```

Output:

MEMBER_ID	NAME	PLAN_NAME	DURATION_MONTHS
1	Rahul	Annual Basic	12
2	Priya	Annual Premium	12
5	Kiran	Half Year Basic	6
6	Ananya	Half Year Premium	6
9	Rohit	Elite Annual	12
15	Karthik	Corporate Annual	12

29. Plans That Collected ₹9,000 or More

```

SELECT pl.plan_name,
       SUM(p.amount) AS total_collected
FROM Payments p
JOIN Memberships ms ON p.membership_id = ms.membership_id
JOIN Membership_Plans pl ON ms.plan_id = pl.plan_id
GROUP BY pl.plan_name
HAVING SUM(p.amount) >= 9000
ORDER BY total_collected DESC, pl.plan_name;

```

Output:

PLAN_NAME	TOTAL_COLLECTED
Elite Annual	15000
Corporate Annual	13000
Annual Premium	12000
Annual Basic	9000
Personal Training	9000

30. View of Active Member Details

```
CREATE OR REPLACE VIEW active_member_details AS
SELECT m.member_id, m.name, pl.plan_name,
       t.trainer_name, ms.end_date
FROM Memberships ms
JOIN Members m ON ms.member_id = m.member_id
JOIN Membership_Plans pl ON ms.plan_id = pl.plan_id
JOIN Trainers t ON ms.trainer_id = t.trainer_id
WHERE ms.status = 'Active';

SELECT * FROM active_member_details
ORDER BY member_id;
```

Output:

MEMBER_ID	NAME	PLAN_NAME	TRAINER_NAME	END_DATE
1	Rahul	Annual Basic	Amit	04-JAN-27
2	Priya	Annual Premium	Neeraj	09-JAN-27
5	Kiran	Half Year Basic	Manoj	31-JUL-26
6	Ananya	Half Year Premium	Shreya	04-AUG-26
8	Neha	Student Quarterly	Nisha	14-MAY-26
9	Rohit	Elite Annual	Sanjay	19-FEB-27
12	Divya	Couple Quarterly	Pallavi	09-JUN-26
13	Suresh	Personal Training	Ramesh	14-JUN-26
15	Karthik	Corporate Annual	Tarun	24-MAR-27

10. CONCLUSION

The Gym Membership Management System provides a structured relational database for managing important gym operations. The system organizes member, trainer, plan, membership, payment, and attendance information into separate relations.

The use of primary keys and foreign keys establishes relationships between the tables and helps maintain data integrity. The normalization process from UNF to 1NF, 2NF, and 3NF reduces redundancy and improves consistency.

The separation of plan details into MEMBERSHIP_PLANS and trainer details into TRAINERS, with MEMBERSHIPS linking them to members, allows each piece of information to be stored once and reused. Separate relations for PAYMENTS and ATTENDANCE allow fee collection and daily presence to be tracked clearly.

SQL queries demonstrate the ability to retrieve, filter, aggregate, and combine information from different relations. Overall, the project demonstrates the practical application of relational database concepts using Oracle Database.

11. FUTURE ENHANCEMENTS

The Gym Membership Management System can be extended in several ways:

1. Online Registration and Payment Gateway

Members could register, choose plans, and pay fees through a web or mobile interface.

2. Automated Renewal Reminders

SMS or e-mail alerts can be sent before a membership expires, and expired memberships can be updated automatically.

3. Biometric / QR-Based Attendance

Attendance can be captured automatically at the gym entrance instead of being entered manually.

4. Diet and Workout Plans

A module can store personalized workout schedules and diet charts for members.

5. Equipment Inventory

A separate module can track gym equipment, maintenance schedules and stock of supplements.

6. Trainer Scheduling and Performance

Training slots and trainer-wise member progress can be added.

7. Role-Based Access

Different database privileges can be provided to administrators, receptionists, and trainers.

8. Graphical User Interface

A frontend application can be connected to the Oracle database for easier interaction.

12. REFERENCES

1. Oracle Database Documentation – Oracle SQL and relational database concepts.
2. Database Management System Course Materials – Concepts related to relational databases, normalization, SQL, keys, and constraints.
3. Oracle SQL Developer – Used for database creation, SQL execution, and query testing.
4. Aditya University DBMS Project Guidelines and Course Material – Used as the academic basis for the project structure and database concepts.
5. Project Database Design and SQL Implementation – Gym Membership Management System developed as part of the DBMS Cornerstone Project.

Reference Books:

1. Database Systems, Ramez Elmasri, Shamkant B. Navathe, 7th Edition, Pearson, ISBN: 9780137504277.
2. Database Systems, Carlos Coronel, Steven Morris, Peter Rob, 10th Edition, Cengage, ISBN: 978-0357517864.
3. Introduction to Database Systems, C J Date, Pearson, 9th Edition, ISBN: 978-0133970777.

Textbooks:

1. Database System Concepts, Abraham Silberschatz, Henry Korth, and S. Sudarshan, McGrawHill, 8th Edition, ISBN-13: 978-1260230508.
2. Database Management Systems, Raghurama Krishnan, Johannes Gehrke, Mc Graw-Hill, 4th Edition, ISBN: 978-1260091195.